A MAJOR PROJECT REPORT ON

AI RESUME ANALYZER AND JOB RECOMMENDATION SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

Submitted by

Aditya Roshan Singh

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Acknowledgement

I would like to express my sincere gratitude to my project guide, [Guide Name], for their invaluable guidance, encouragement, and support throughout the development of this project. Their insights into both the technical and practical aspects of building a real-world AI system were instrumental in shaping this work.

I am also thankful to the Head of the Department and all the faculty members of the Department of Computer Science & Engineering for providing the resources and environment necessary to complete this project.

Finally, I would like to thank my family and friends for their constant support and motivation during the course of this project.

Abstract

Job seekers frequently struggle to understand why their resume fails to secure interviews, particularly when applying to roles with specific and varied skill requirements. Manually comparing a resume against a job description is time-consuming and error-prone, and most candidates lack visibility into which of their skills are being effectively communicated versus which important skills are missing entirely.

This project presents the AI Resume Analyzer and Job Recommendation System, a Python-based application that automates this comparison process. The system accepts a resume in PDF or DOCX format, extracts and cleans its textual content, and identifies the skills present using a curated skill dictionary. It then applies Term Frequency-Inverse Document Frequency (TF-IDF) vectorization together with Cosine Similarity to quantitatively compare the resume against a set of predefined job roles, producing a ranked list of the best-fitting roles along with a percentage match score.

For the top-matching role, the system performs a skill-gap analysis, identifying which of the role's required skills are absent from the candidate's resume, and generates a personalized learning roadmap suggesting resources for each missing skill. Finally, the system compiles all findings into a downloadable Word document report, giving the user a complete, actionable diagnosis of their resume's fit for a target role.

The system was built using Python, Streamlit for the web interface, Pandas and NumPy for data handling, Scikit-learn for the TF-IDF and cosine similarity computations, and PyPDF/python-docx for file parsing and report generation. The result is a lightweight, explainable, and easily deployable tool that directly addresses a common and practical problem faced by job seekers.

Table of Contents

Chapter 1: Introduction

1.1 Background

The modern job market is highly competitive, and Applicant Tracking Systems (ATS) as well as human recruiters often skim resumes for specific keywords and quantifiable achievements. Many capable candidates fail to advance in hiring pipelines not due to a lack of skill, but due to a poorly structured or poorly targeted resume. This creates a clear opportunity for a tool that can objectively evaluate a resume against a specific role and highlight concrete areas of improvement.

1.2 Problem Statement

Given a resume and a set of job roles with their associated required skills, there is no simple, automated way for a candidate to determine: (a) which job role their current resume is best suited for, (b) what specific skills are missing for that role, and (c) what actionable steps they should take to close that gap. This project aims to solve this problem using Natural Language Processing (NLP) and classical machine learning similarity techniques.

1.3 Objectives

- To design a system that can extract and process text from resumes in PDF and DOCX formats.
- To build a skill-detection mechanism using a curated skill dictionary.
- To implement a similarity-scoring engine using TF-IDF and Cosine Similarity to rank job roles by fit.
- To identify missing skills for the best-matching role and generate relevant learning suggestions.
- To provide a simple, interactive web interface using Streamlit.
- To generate a downloadable, well-formatted report summarizing the entire analysis.

1.4 Scope of the Project

The current scope of the system is limited to a predefined set of job roles and skills stored in CSV files, and supports resumes in PDF and DOCX formats only. The matching algorithm is keyword/frequency-based (TF-IDF) rather than based on deep semantic understanding, which keeps the system fast, lightweight, and easily explainable — an important consideration for an academic project. The system is designed to be extended in the future with more advanced NLP models, a larger skill/role database, and cloud deployment, as discussed in Chapter 8.

1.5 Organization of the Report

This report is organized as follows: Chapter 2 discusses existing systems and their limitations. Chapter 3 covers the system requirements and feasibility analysis. Chapter 4 presents the system design, including architecture and algorithms. Chapter 5 details the implementation of each module. Chapter 6 describes the testing strategy and test cases. Chapter 7 presents the results and discussion. Chapter 8 concludes the report and outlines future scope.

Chapter 2: Literature Review

2.1 Existing Systems

Several commercial tools currently address parts of the resume-analysis problem. Applicant Tracking System (ATS) checkers, offered by platforms such as Jobscan and Resume Worded, compare a resume against a job description and provide a compatibility score. LinkedIn's "Skills Match" feature highlights overlapping skills between a candidate's profile and a job posting. Grammar and formatting tools such as Grammarly focus on language quality rather than content-role fit.

2.2 Limitations of Existing Systems

- Most commercial tools are proprietary, subscription-based, and offer no visibility into how the matching score is actually computed.
- Existing tools rarely provide a structured, downloadable learning roadmap tailored to the specific skill gaps identified.
- Many tools are tightly coupled to a single job description at a time and do not rank a resume against multiple roles simultaneously.
- Few tools combine skill-gap detection with actionable next steps in a single, unified report.

2.3 Proposed System

The proposed AI Resume Analyzer addresses these gaps by using a transparent, explainable algorithm (TF-IDF and Cosine Similarity) whose behavior can be fully understood and audited, unlike black-box commercial alternatives. It ranks a resume against multiple job roles at once, explicitly lists missing skills, and generates a personalized roadmap alongside a downloadable report — combining diagnosis and actionable guidance in a single lightweight, open, and extensible application.

Chapter 3: System Analysis

3.1 Functional Requirements

- The system shall allow users to upload a resume in PDF or DOCX format.
- The system shall extract and clean the text content of the uploaded resume.
- The system shall detect known skills present in the resume text.
- The system shall compute a similarity score between the resume and each available job role.
- The system shall display the top 3 best-matching job roles ranked by score.
- The system shall identify missing skills for the best-matching role.
- The system shall generate learning suggestions for each missing skill.
- The system shall allow the user to download a Word document summarizing the full analysis.

3.2 Non-Functional Requirements

- Performance: The system should return analysis results within a few seconds for a typical resume.
- Usability: The interface should be simple enough for a non-technical user to operate without instructions.
- Portability: The system should run on any machine with Python installed, independent of the operating system.
- Maintainability: The codebase should be modular, with each responsibility isolated into its own file.

3.3 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core 2 GHz or higher
RAM	4 GB (8 GB recommended)
Storage	500 MB free disk space
Internet	Required only for initial package installation

3.4 Software Requirements

Category	Technology Used
Programming Language	Python 3.10+
Web Framework	Streamlit
Data Handling	Pandas, NumPy
Machine Learning	Scikit-learn (TF-IDF Vectorizer, Cosine Similarity)
File Parsing	PyPDF, python-docx
Report Generation	python-docx
IDE	VS Code / PyCharm
Operating System	Windows / macOS / Linux

3.5 Feasibility Study

Technical Feasibility: All libraries used (Streamlit, Pandas, Scikit-learn, PyPDF, python-docx) are open-source, well-documented, and require no specialized hardware such as a GPU, making the project technically feasible on standard consumer hardware.

Economic Feasibility: The entire technology stack is free and open-source. There are no licensing costs, and free-tier deployment platforms (such as Streamlit Community Cloud) are sufficient for hosting a working demonstration, making the project economically feasible for a student project.

Operational Feasibility: The system requires no specialized training to operate — a user simply uploads a resume and reads the results, making it operationally practical for its intended non-technical audience.

Chapter 4: System Design

4.1 System Architecture

The system follows a linear pipeline architecture, where the output of each stage becomes the input to the next. This design was chosen for its simplicity and ease of debugging — each stage can be tested independently.

Stage	Input	Output
1. Upload	Resume file (PDF/DOCX)	File saved to disk
2. Text Extraction	File path	Raw resume text
3. Text Cleaning	Raw text	Cleaned, lowercase text
4. Skill Extraction	Cleaned text + skill dictionary	List of detected skills
5. Matching	Resume text + job roles CSV	Ranked job roles with match scores
6. Gap Analysis	Detected skills + required skills	List of missing skills
7. Roadmap Generation	Missing skills	Learning suggestions
8. Report Generation	All of the above	Downloadable .docx report

4.2 Module Description

Module (File)	Responsibility
app.py	Streamlit UI; orchestrates all other modules
resume_parser.py	Extracts raw text from PDF/DOCX files
text_cleaner.py	Normalizes text for NLP processing
skill_extractor.py	Detects known skills in resume text
matcher.py	Computes TF-IDF vectors and cosine similarity scores
roadmap_generator.py	Maps missing skills to learning resources
report_generator.py	Builds the downloadable Word report
utils.py	Shared helper functions (CSV loading, path handling)

4.3 Data Flow Diagram (Conceptual)

Resume File → Text Extraction → Text Cleaning → Skill Detection → Similarity Matching (against Job Roles dataset) → Best Role + Missing Skills → Roadmap → Final Report. Each arrow represents a function call in app.py, and each stage's output is passed directly as the next stage's input, with no persistent database required.

4.4 Core Algorithm: TF-IDF and Cosine Similarity

Term Frequency-Inverse Document Frequency (TF-IDF) is a numerical statistic that reflects how important a word is to a document within a collection of documents (corpus). It increases proportionally with the number of times

a word appears in a document, but is offset by how frequently the word appears across all documents, which helps filter out common but unimportant words.

TF-IDF(t, d) = TF(t, d) × IDF(t), where TF(t, d) is the frequency of term t in document d, and IDF(t) = log(N / df(t)), with N being the total number of documents and df(t) the number of documents containing term t.

Once both the resume and a job role's description are converted into TF-IDF vectors, Cosine Similarity is used to measure how similar the two vectors are, based on the cosine of the angle between them:

Cosine Similarity(A, B) = (A · B) / (||A|| × ||B||)

A score close to 1 indicates high similarity (strong keyword overlap, weighted by importance), while a score close to 0 indicates little to no overlap. This score is scaled to a percentage (0-100%) for easier interpretation by the end user.

Chapter 5: Implementation

5.1 Technology Stack Summary

The project is implemented entirely in Python 3, chosen for its rich ecosystem of NLP and data-science libraries, ease of learning, and strong community support. Streamlit was selected for the front-end because it allows a fully interactive web interface to be built with minimal code, letting the project focus on the AI logic rather than web development boilerplate.

5.2 Resume Parsing (resume_parser.py)

This module detects the uploaded file's extension and routes it to the appropriate parser: pypdf's PdfReader for PDF files, iterating page-by-page and concatenating extracted text, or python-docx's Document class for DOCX files, iterating through paragraphs. If no text can be extracted (for example, from a scanned image-based PDF with no real text layer), the module raises a clear, descriptive error rather than failing silently.

5.3 Text Cleaning (text_cleaner.py)

Raw extracted text is noisy — it contains punctuation, numbers, inconsistent casing, and filler words that add no discriminative value to the similarity calculation. This module lowercases the text, strips out non-alphabetic characters using regular expressions, and removes common English stopwords using Scikit-learn's built-in stopword list, producing a clean, normalized string ready for vectorization.

5.4 Skill Extraction (skill_extractor.py)

The skill dictionary (data/skill_dictionary.csv) contains a curated list of common technical and soft skills across categories such as programming, web development, data science, cloud computing, and databases. This module checks the cleaned resume text for the presence of each skill as a substring, building a list of skills the candidate appears to possess. This same module also provides a helper to compute the difference between a role's required skills and the candidate's detected skills.

5.5 Matching Engine (matcher.py)

This is the core AI module. It uses Scikit-learn's TfidfVectorizer to convert both the resume text and each job role's combined text (role name + description + required skills) into TF-IDF vectors, and then applies cosine_similarity from sklearn.metrics.pairwise to compute a similarity score between them. The recommend_job_roles function repeats this process for every row in job_roles.csv, attaches the resulting score to each role, and returns the top N roles sorted in descending order of match score.

5.6 Roadmap Generation (roadmap_generator.py)

For each missing skill identified in the gap analysis, this module looks up a curated dictionary mapping skill names to recommended learning resources (e.g. official documentation, well-known free courses). If a specific skill is not present in this lookup dictionary, a sensible generic fallback suggestion is returned instead, ensuring the user always receives actionable guidance.

5.7 Report Generation (report_generator.py)

Using the python-docx library, this module programmatically builds a Word document containing the best-matching role and its score, the list of detected skills, the list of missing skills, and the generated learning roadmap, formatted with headings and bullet lists. The resulting file is saved to the reports/ folder and made available to the user as a download through the Streamlit interface.

5.8 User Interface (app.py)

app.py is the single entry point of the application, run using the command streamlit run app.py. It presents a file uploader restricted to PDF and DOCX formats, and upon upload, sequentially invokes each module described above, displaying the results — detected skills, ranked job roles, missing skills, and the learning roadmap — directly in the browser, along with a button to generate and download the final report.

Chapter 6: Testing

6.1 Testing Strategy

The system was tested using a combination of unit-level checks on individual modules (e.g. verifying text_cleaner.py correctly strips punctuation) and end-to-end integration testing, where a sample DOCX resume was passed through the entire pipeline, from parsing to final report generation, to confirm that data flows correctly between all modules and that the final output is sensible.

6.2 Test Cases

Test Case	Input	Expected Output	Result
Valid PDF upload	A well-formatted PDF resume	Text extracted successfully	Pass
Valid DOCX upload	A well-formatted DOCX resume	Text extracted successfully	Pass
Unsupported file type	A .txt file	Clear 'Unsupported file type' error	Pass
Scanned/image PDF	PDF with no real text layer	'No text could be extracted' error	Pass
Skill detection accuracy	Resume listing Python, SQL, Excel	All three skills correctly detected	Pass
Similarity scoring	Resume closely matching a job description	High match score (>70%) for that role	Pass
Skill-gap detection	Resume missing one required skill	That skill correctly listed as missing	Pass
Report generation	Completed analysis	Valid, well-formatted .docx file produced	Pass
CSV path resolution	App run from a different working directory	CSV files still load correctly	Pass (after fix)

6.3 Observations

An issue was identified during testing where CSV file paths were resolved relative to the terminal's current working directory rather than the project's root folder, causing a FileNotFoundError when the application was launched from outside the project directory. This was resolved by rewriting the path resolution logic in utils.py to compute an absolute path based on the file's own location, ensuring consistent behavior regardless of where the application is launched from.

Chapter 7: Results and Discussion

7.1 Sample Output

A sample resume listing skills such as Python, SQL, Machine Learning, Pandas, NumPy, Data Analysis, and Excel was passed through the complete pipeline. The system correctly extracted all seven skills and ranked the available job roles as follows:

Rank	Job Role	Match Score
1	Data Analyst	37.77%
2	Data Scientist	34.07%
3	Machine Learning Engineer	23.87%

For the top-ranked role, Data Analyst, the system correctly identified "Power BI" and "Data Visualization" as missing skills, and generated corresponding learning suggestions for both, demonstrating that the full pipeline — from parsing through to roadmap generation — functions correctly end to end.

7.2 Advantages of the System

- Transparent and explainable matching logic, unlike black-box commercial alternatives.
- Fast execution, with no need for GPU acceleration or large pre-trained models.
- Modular codebase that is easy to extend, test, and maintain.
- Produces a concrete, downloadable deliverable (the report) rather than just an on-screen score.

7.3 Limitations

- TF-IDF and cosine similarity rely on keyword overlap and do not capture deeper semantic meaning — for example, "led a team" and "managed a group" would not be recognized as similar.
- Skill detection uses substring matching, which can occasionally produce false positives or miss skills phrased differently than in the dictionary.
- The system currently supports a fixed, CSV-based set of job roles rather than arbitrary, user-provided job descriptions.

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

This project successfully demonstrates an end-to-end AI-based resume analysis pipeline that parses resumes, detects skills, computes similarity scores against job roles using TF-IDF and Cosine Similarity, identifies skill gaps, and produces an actionable, downloadable report. The system addresses a genuine, practical problem faced by job seekers, and does so using a transparent and easily explainable methodology suitable for both academic demonstration and further real-world development.

8.2 Future Enhancements

- Replace TF-IDF with semantic sentence embeddings (e.g. using the sentence-transformers library) to capture meaning rather than exact keyword overlap.
- Allow users to paste an arbitrary job description directly, rather than being limited to a fixed CSV of job roles.
- Integrate a large language model (LLM) to rewrite weak resume bullet points into stronger, quantified statements.
- Migrate from CSV files to a relational database such as PostgreSQL to support multiple users and historical tracking.
- Add user authentication so candidates can track their resume's improvement over time.
- Deploy the system using a scalable architecture (e.g. FastAPI backend with a React frontend) for production-grade use as a public-facing web product.

References

1. Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 2011.
2. Streamlit Documentation. Available at: https://docs.streamlit.io
3. Scikit-learn Documentation — TfidfVectorizer and Cosine Similarity. Available at: https://scikit-learn.org
4. pypdf Documentation. Available at: https://pypdf.readthedocs.io
5. python-docx Documentation. Available at: https://python-docx.readthedocs.io
6. Manning, C. D., Raghavan, P., and Schütze, H. "Introduction to Information Retrieval." Cambridge University Press, 2008.